



Tarefa 3

UNICAMP

Padrões de Projeto, Interfaces e Enumerações

INSTITUTO DE
COMPUTAÇÃO

MC322 - Programação Orientada a Objetos

Professor: Marcelo da Silva Reis

PEDs: Athyrson Ribeiro, Matheus Lozano

PADs: João Pedro Frare

1 Descrição Geral

A planta industrial automatizada desenvolvida nas tarefas anteriores expandiu suas operações e agora lida com múltiplos fluxos de produção, demandas concorrentes e auditorias de qualidade.

Com esse crescimento, surgiram dois novos desafios arquiteturais:

1. **Flexibilidade de Produção:** Diferentes cenários operacionais exigem critérios distintos para priorizar e selecionar quais demandas devem ser fabricadas a seguir.
2. **Padronização e Segurança de Tipos:** O uso de textos puros (**String**) para estados e prioridades expõe o sistema a erros de digitação e comportamentos inconsistentes, enquanto componentes de diferentes origens necessitam de contratos padronizados para auditoria e relatórios.

Nesta **Tarefa 3**, refatoraremos a aplicação aplicando o padrão de projeto **Strategy**, contratos com **Interfaces múltiplas** e **Enumerações (Enums)**. Além disso, o sistema deverá suportar **dois cenários de operação** (Ideal e Apocalíptico), selecionáveis no início da execução, e oferecer uma **visualização detalhada do armazém de produtos acabados**.

1.1 Evolução da Planta

A planta agora deve:

- Encapsular algoritmos de seleção de demanda utilizando o padrão **Strategy**;
- Permitir a troca de estratégia de produção **dinamicamente em tempo de execução**;
- Substituir representações baseadas em texto por **Enumerações (enum)**;
- Implementar uma interface de diagnósticos e auditoria (**Auditavel**) em múltiplas classes da hierarquia;
- Manter o desacoplamento do **GerenciadorProducao** em relação às implementações concretas;
- Suportar **dois cenários de operação** (Ideal e Apocalíptico) com parâmetros distintos de orçamento e confiabilidade;
- Modelar o **desgaste progressivo das máquinas** a cada uso;
- Expor o **estado dos produtos no armazém** (quantidades, qualidade, lotes).

1.2 Novos Conceitos

1. **Padrão Strategy:** Interface **EstrategiaProducao** com pelo menos 3 implementações concretas;
2. **Enumerações (Enums):** Uso de **enum** para representar estados de demandas e máquinas com tipagem forte;
3. **Polimorfismo com Interfaces Múltiplas:** Uso de interfaces para padronizar comportamentos transversais (auditoria/relatório);
4. **Desacoplamento e SRP:** O **GerenciadorProducao** delega decisões de priorização para o objeto estratégia atual;
5. **Cenários de Simulação:** Configurações pré-definidas que alteram orçamento inicial e probabilidades de falha.

2 Objetivos

- Compreender o propósito e a aplicação de Padrões de Projeto GoF (Behavioral Patterns);
- Implementar e utilizar o padrão [Strategy](#);
- Utilizar `enum` em Java para garantir integridade de dados e métodos associados a constantes;
- Explorar interfaces como contratos de capacidades transversais entre classes não parentes;
- Refatorar o sistema existente mantendo retrocompatibilidade sem duplicação de lógica (Princípio DRY);
- Modelar desgaste e falhas de forma probabilística, refletindo cenários operacionais distintos.

3 Atividades

3.1 Implementação e Refatoração de Classes

3.1.1 Enumeração `StatusDemanda`

Para evitar o uso de textos soltos, o estado de uma demanda deve ser gerido exclusivamente por uma enumeração `StatusDemanda`.

Valores Obrigatórios

- `PENDENTE` – Demanda cadastrada, mas ainda não iniciada;
- `EM_PRODUCAO` – Demanda selecionada e em processo de fabricação;
- `CONCLUIDA` – Demanda com todos os produtos produzidos com sucesso;
- `CANCELADA` – Demanda cancelada por falta de orçamento ou insumos.

Métodos Recomendados

- `getDescricao` – Retorna uma descrição formatada do status para o menu/relatório.

3.1.2 Interface `Auditavel`

Para demonstrar a utilidade de interfaces além do padrão `Strategy`, deve ser criada a interface `Auditavel`. Ela representa qualquer componente do sistema capaz de fornecer um diagnóstico operacional detalhado para relatórios de auditoria da fábrica.

Métodos da Interface

- `gerarRelatorioDiagnostico()` – Retorna uma `String` com o estado de saúde/qualidade do componente;
- `precisaManutencao()` – Retorna um `boolean` indicando se o componente requer intervenção.

Classes que devem implementar `Auditavel`:

- `Maquina` (e por consequência suas subclasses): verifica probabilidade de falha acumulada ou histórico de erros;
- `Produto` (e por consequência suas subclasses): verifica a qualidade e o nível de risco acumulado durante a produção.

3.1.3 Interface `EstrategiaProducao`

Interface que define o contrato para os algoritmos de seleção da próxima demanda a ser atendida pela fábrica.

Métodos da Interface

- `selecionarDemanda(List<Demanda> demandas, double orcamentoDisponivel)` – Examina a lista de demandas disponíveis e retorna a referência da melhor `Demanda` a ser processada, ou `null` caso nenhuma seja elegível.
- `getNomeEstrategia()` – Retorna o nome amigável da estratégia para exibição na interface do usuário.

3.1.4 Subclasses / Implementações Concretas de `EstrategiaProducao`

Importante: Os alunos **devem adaptar os nomes e os critérios** das estratégias para torná-los coerentes com o tema escolhido para a fábrica (ver Seção 4). Abaixo estão os requisitos mínimos para as 3 implementações obrigatórias:

1. `EstrategiaOrdemChegada` – Seleciona a primeira demanda cadastrada com status `PENDENTE` (Fila First-In, First-Out).
2. `EstrategiaMaiorDemanda` – Prioriza a demanda que possui a maior quantidade total de itens a serem fabricados.
3. `EstrategiaMaximoProdutos` – Examina o orçamento disponível e seleciona a demanda viável que maximiza o número total de produtos finais produzidos sem estourar o *budget*.

3.1.5 Refatoração da Classe `Demanda`

A classe `Demanda` desenvolvida na Tarefa 2 deve ser modificada:

- Alterar o atributo `atendida` (boolean) para `status` do tipo `StatusDemanda`;
- Atualizar os métodos de transição de estado para garantir integridade (ex: não é possível concluir uma demanda `CANCELADA`);
- Adicionar métodos de consulta de viabilidade financeira e consumos estimados.

3.1.6 Refatoração da Classe `GerenciadorProducao`

A classe `GerenciadorProducao` deixa de ter uma regra fixa de seleção de demandas e passa a atuar como o **Contexto** do padrão Strategy.

Novos Atributos

- `estrategiaAtual` – Referência para o tipo da interface `EstrategiaProducao`.

Novos Métodos / Alterações

- `setEstrategia(EstrategiaProducao novaEstrategia)` – Permite a alteração dinâmica da estratégia em tempo de execução;
- `executarProximaProducao()` – Utiliza `estrategiaAtual.selecionarDemanda(...)` para identificar a demanda correta e inicia a fabricação;
- `gerarAuditoriaGeral()` – Percorre coleções de objetos que implementam `Auditavel` e exibe um relatório consolidado da planta;
- `exibirArmazem()` – Lista todos os produtos acabados em estoque, com quantidade, qualidade e lote.

3.1.7 Desgaste Progressivo das Máquinas

A saúde (*health*) de cada máquina deve ser um valor entre 0 e 100. A cada uso (ou seja, a cada ciclo de fabricação em que a máquina for empregada):

- A saúde da máquina deve sofrer uma **pequena redução aleatória** (ex: entre 0 e 3 pontos), simulando o desgaste natural;
- A probabilidade de falha durante a produção deve ser inversamente proporcional à saúde atual (máquinas mais saudáveis falham menos);
- Ao atingir saúde 0 (ou um limiar crítico), a máquina deve ser considerada **quebrada** e não pode mais operar até ser reparada (reparo pode ser implementado como método extra, se desejado);
- O método `precisaManutencao()` deve retornar `true` quando a saúde estiver abaixo de um limiar configurável (ex: 30).

Livre para adicionar métodos como `reparar()`, `getSaude()`, `setSaude()`, ou atributos como `historicoFalhas`, conforme a necessidade do projeto.

3.1.8 Cenários de Operação

O sistema deve oferecer, **no início da execução**, uma escolha entre dois cenários:

1. Cenário Ideal:

- Orçamento (budget) **farto**;
- Máquinas e produtos com **baixa probabilidade de falha**;
- Desgaste das máquinas reduzido;
- Cenário voltado para demonstrar o fluxo completo sem estresse financeiro.

2. Cenário Apocalíptico:

- Orçamento (budget) **apenas para o básico**;
- Máquinas e produtos com **alta probabilidade de falha**;
- Desgaste das máquinas acelerado;
- Cenário voltado para testar a robustez das estratégias e do sistema sob pressão.

Sugestão de implementação: crie uma classe `ConfiguracaoCenario` ou um `enum Cenario` com os parâmetros de cada cenário. O usuário escolhe no menu inicial, e o `GerenciadorProducao` é inicializado com os valores correspondentes.

3.1.9 Visualização do Armazém

O sistema deve permitir ao usuário **consultar o estado dos produtos no armazém**. Essa visualização deve mostrar, no mínimo:

- Nome/tipo de cada produto armazenado;
- Quantidade disponível;
- Qualidade atual (se aplicável);
- Lote de produção (se aplicável);
- Status de risco (`Auditavel.precisaManutencao()` ou equivalente).

A visualização pode ser via menu (`Ver armazém`) e deve ser distinta da consulta de **matéria-prima** (que lista insumos brutos, não produtos acabados).

3.2 Menu do Sistema

O menu principal deve ser **claro e organizado**, com seções bem definidas. É esperado que o grupo vá além do exemplo abaixo, adicionando **submenus** para agrupar funcionalidades relacionadas (ex: um submenu de “Fabricar”, um submenu de “Consultar”, um submenu de “Auditoria”).

O menu deve destacar a **Estratégia Atual** ativa, o **Budget Atual** e o **Cenário Ativo**, e permitir o acesso rápido à troca de estratégias, relatórios de auditoria e visualização do armazém.

Exemplo de menu (adaptar ao tema do grupo):

```
=====
|                               [FABRICA DE PAES ARTESANAIS]                               |
| ESTRATEGIA ATUAL: [Ordem de Chegada]                                                  |
| CENARIO ATIVO:      [Ideal]                                                            |
| BUDGET ATUAL:       R$ 10000.00                                                        |
=====

-----
[ATUALIZAR DEMANDAS]
-----
1 - Atualizar demanda de Pao Artesanal
2 - Atualizar demanda de Pao de Forma
3 - Atualizar demanda de Pao Simples

-----
[FABRICAR]
-----
4 - Fabricar Pao Artesanal
5 - Fabricar Pao de Forma
6 - Fabricar Pao Simples

-----
[CONSULTAR]
-----
7 - Ver armazem (produtos acabados)
8 - Ver estoque de materia-prima

-----
[COMPRAR MATERIA-PRIMA]
-----
9 - Comprar materia-prima

-----
[GERENCIAMENTO DE ESTRATEGIA]
-----
10 - Alterar Estrategia de Producao
      (1: Ordem de Chegada | 2: Maior Demanda | 3: Maximizar Producao)

-----
[AUDITORIA]
-----
11 - Executar Relatorio de Auditoria (Interface Auditavel)

-----
0 - SAIR

ESCOLHA:
```

Observação: os itens acima podem (e devem) ser reorganizados em **submenus** para melhorar a usabilidade. Exemplo de estrutura sugerida:

```
MENU PRINCIPAL
1. Demandas
```

- Atualizar demanda
- Listar demandas (status via enum)
- Voltar
- 2. Fabricacao
 - Processar proxima demanda (usa estrategia ativa)
 - Fabricar item especifico
 - Voltar
- 3. Consultar
 - Ver armazem (produtos acabados)
 - Ver estoque de materia-prima
 - Voltar
- 4. Comprar materia-prima
- 5. Gerenciar estrategia
 - Ordem de Chegada
 - Maior Demanda
 - Maximizar Producao
 - Voltar
- 6. Auditoria
 - Relatorio geral
 - Detalhar maquinas
 - Detalhar produtos
 - Voltar
- 0. Sair

3.3 Pontos de Atenção

- **Polimorfismo:** `GerenciadorProducao` deve depender **apenas** da interface `EstrategiaProducao`, nunca das classes concretas das estratégias;
- **Troca em Tempo de Execução:** O usuário deve conseguir trocar de estratégia no menu a qualquer momento e ver o impacto imediato na ordem de produção;
- **Enums Obrigatórios:** O uso do enum `StatusDemanda` é obrigatório para controlar os estados da demanda;
- **Uso da Interface Auditavel:** O relatório de auditoria deve demonstrar o uso de polimorfismo iterando sobre instâncias de `Auditavel` (produtos e máquinas);
- **Encapsulamento:** Mantenha os atributos privados com getters/setters adequados;
- **Extensibilidade:** É permitido (e encorajado) **adicionar novos métodos e atributos** às classes existentes, desde que mantida a coerência arquitetural e o encapsulamento. Não se limite ao estritamente listado neste enunciado.

4 Criatividade e Evolução do Tema

Esta seção detalha como os critérios de criatividade serão avaliados. Eles são parte integrante da nota e devem ser levados a sério.

- **Nomes coerentes com o tema:** Todas as classes, interfaces, enums, métodos e atributos devem ter nomes que reflitam o tema escolhido pela dupla para a fábrica. Por exemplo, se o tema é “Fábrica de Pães Artesanais”, as classes devem se chamar `Massa`, `FornoSecador`, `Receita`, etc. — e não nomes genéricos como `ProdutoA`, `MaquinaX` ou `Item1`. Nomes genéricos podem ser utilizados, mas com cautela.
- **Regras de negócio adaptadas:** As 3 estratégias de produção devem fazer sentido para o tema. Ex: em uma fábrica de pães, “Maior Demanda” pode virar “Estratégia do Lote Família” (prioriza a demanda que alimenta mais pessoas).
- **Estilo visual do menu:** O menu conta pontos. Um menu **claro, organizado, com seções bem definidas, cores (se o terminal suportar), alinhamento e uso consistente de separadores** será valorizado. Menus bagunçados, com opções repetidas ou sem hierarquia, serão penalizados.
- **Submenus:** A organização em submenus é esperada e demonstra cuidado com a experiência do usuário.

- **Toque humano e originalidade:** Elementos como mensagens temáticas, nomes criativos para eventos (ex: “A massa passou do ponto”), ou pequenas narrativas ao longo da execução contam pontos.

5 Avaliação

Importante: A avaliação desta tarefa **não se limita** aos itens abaixo. **Todo o enunciado** (descrição geral, objetivos, atividades, pontos de atenção, criatividade, boas práticas e entrega) será utilizado como base para a correção. Os itens a seguir são apenas **diretrizes gerais** para orientar o desenvolvimento, não uma lista exaustiva.

1. Implementação do Padrão Strategy e Interfaces(2,0pts)

- Interface `EstrategiaProducao` corretamente declarada;
- Pelo menos 3 estratégias concretas coerentes com o tema;
- Troca dinâmica de estratégia em tempo de execução sem alterar o gerenciador;
- Interface `Auditavel` aplicada corretamente a classes de hierarquias distintas.

2. Uso de Enums e Refatoração de Fluxo(1,0pt)

- Substituição correta de flags/Strings por `enum StatusDemanda`;
- Lógica de transição de status consistente;
- Integração fluida entre o fluxo de fabricação e as estratégias.

3. Cenários e Simulação(1,5pts)

- Implementação dos cenários Ideal e Apocalíptico com parâmetros distintos;
- Seleção do cenário no início da execução;
- Desgaste progressivo das máquinas a cada uso;
- Impacto visível do cenário no orçamento e nas taxas de falha.

4. Visualização e Auditoria(2,0pts)

- Menu para consultar o armazém de produtos acabados;
- Distinção clara entre armazém (produtos) e estoque (matéria-prima);
- Relatório de auditoria demonstrando polimorfismo sobre `Auditavel`.

5. Organização e Documentação do Código(1,5pts)

- Cada classe, interface e enum em arquivo `.java` próprio;
- Código limpo, sem duplicação (DRY) e bem comentado;
- Separação clara de responsabilidades (SRP).

6. Criatividade e Evolução do Tema(2,0pts)

- Nomes e regras de negócio adaptados ao tema da fábrica;
- Menu do console claro, informativo, com submenus e bom estilo visual;
- Atualização completa do `justificativa.txt`;
- Toque humano e originalidade.

7. Penalidade: Não compilação = -5,0 pontos.

6 Entrega

1. **GitHub:** Release denominada **tarefa3-RA1-RA2** contendo o código fonte completo e o arquivo `justificativa.txt`;
2. **Google Forms:** Formulário preenchido com nomes, RAs e link direto da release no GitHub.

Atenção: O formulário do Google Forms **só deve ser preenchido após a dupla ter criado o release da tarefa no GitHub**. O link da release é um campo obrigatório do formulário; portanto, preencher o formulário antes de criar o release resultará em uma submissão inválida. Recomenda-se:

1. Finalizar o código e testar a compilação;
2. Criar a release **tarefa3-RA1-RA2** no GitHub com todo o código fonte e o `justificativa.txt`;
3. Copiar o link direto da release;
4. Só então preencher o Google Forms com os dados da dupla e o link.

O prazo final de submissão é o dia **25/09/2026 às 23h59**. Entregas atrasadas sofrerão penalização de **25% sobre a nota obtida**. Não serão aceitas entregas efetuadas após **02/10/2026**.

6.1 Compilação e Execução

Para garantir que a estrutura de pacotes e arquivos seja compilada corretamente, utilize:

```
javac -d bin $(find src -name "*.java")
java -cp bin Main
```

7 Dicas de Boas Práticas

- **Padrão Strategy:** Mantenha as classes de estratégia puras. Elas devem apenas decidir qual demanda escolher, sem alterar diretamente o orçamento ou modificar o estoque.
- **Type Safety com Enums:** Evite comparações com `String` usando `.equals()`. Utilize `==` com enums para garantir verificações rápidas e seguras em tempo de compilação.
- **DRY (Don't Repeat Yourself):** Se duas estratégias compartilham filtros comuns (ex: ignorar demandas concluídas), isolar a verificação em um método utilitário privado ou padrão da interface.
- **KISS e YAGNI:** Mantenha as interfaces concisas com apenas os métodos estritamente necessários.
- **Probabilidade e Reprodutibilidade:** Considere usar `java.util.Random` com uma *seed* configurável para permitir testes reproduzíveis, especialmente ao comparar cenários.
- **Submenus:** Agrupe funcionalidades relacionadas. Um menu com 10+ opções planas é difícil de navegar; submenus melhoram a experiência.